SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	Assignment Type: Lab	Academic Year:2025-2026

Course Coordinator Name	Dr. Rishabh Mittal
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju

CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week10 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
NAME:	Tejomayi	**Holl.no:**	2303A54064

Assignment Number:19.3(Present assignment number)/**24**(Total number of assignments)

Q.No.	Question	*Expected Time to complete*
1	**Lab 19 – Code Translation: Converting Between Programming Languages** **Lab Objectives:**	Week10 - Wednesday

- Understand how AI tools can assist in translating code between different programming languages.
- Learn to verify correctness and functionality after translation.
- Explore syntactic and semantic differences between languages (e.g., Python, Java, C++).
- Practice debugging and optimizing AI-translated code.

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:
  - Constructors
  - Data types
  - Access specifiers
- Compile and run both versions to verify output consistency

Expected Output:

An equivalent working C++ class translated from Python.

ai_ass.19.3 > task.01.py > ...

```python
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    # Method to display student details
    def display(self):
        print("Student Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)

    # Method to check pass/fail
    def is_pass(self):
        if self.marks >= 40:
            return True
        return False


# Example usage
s1 = Student("Uday", 20, 85)

s1.display()

if s1.is_pass():
    print("Result: Pass")
else:
    print("Result: Fail")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
Memoized: 832040, Time: 0.00047 seconds
...
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\User
python.exe "c:/Users/MYPC/OneDrive/Desktop/ai assisted coding/a
Student Name: Uday
Age: 20
Marks: 85
Result: Pass
```

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.

ai_ass.19.3 > task.02.py > ...

```python
# prime_check.py

def is_prime(num):
    """Return True if number is prime, otherwise False"""

    if num <= 1:
        return False

    # check divisibility up to square root
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False

    return True


# Testing with multiple inputs
test_numbers = [1, 2, 7, 10, 17, 25, 29]

for n in test_numbers:
    if is_prime(n):
        print(f"{n} is Prime")
    else:
        print(f"{n} is Not Prime")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData
python.exe "c:/Users/MYPC/OneDrive/Desktop/ai assisted coding/ai_ass.19.3/tas
7 is Prime
10 is Not Prime
17 is Prime
25 is Not Prime
29 is Prime
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding>
```

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:
A working Python program that correctly implements the pseudocode logic.

Task 4 – SQL to Pandas Query
Task:
Translate a **SQL query** into a **Pandas DataFrame operation** in Python.
Instructions:
- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:
- Equivalent Pandas query that retrieves the correct subset of data.

ai_ass.19.3 > task.04.py > ...

```python
import pandas as pd

# Creating sample employee data
data = {
    "name": ["Uday", "Rahul", "Anita", "Sneha", "Kiran"],
    "age": [22, 30, 27, 25, 35],
    "salary": [45000, 60000, 52000, 48000, 75000]
}

employees = pd.DataFrame(data)

print("Original DataFrame:")
print(employees)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\Ap
python.exe "c:/Users/MYPC/OneDrive/Desktop/ai assisted coding/ai_ass.19.
    name  age  salary
0   Uday   22   45000
1  Rahul   30   60000
2  Anita   27   52000
3  Sneha   25   48000
4  Kiran   35   75000
```

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
    - Syntax differences
    - Library support
    - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

ai_ass.19.3 > task.05.py > ...

```python
# linear_search.py

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i   # return index
    return -1


# Test execution
numbers = [10, 25, 30, 45, 50]
key = 30

result = linear_search(numbers, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

TERMINAL

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> &
python.exe "c:/Users/MYPC/OneDrive/Desktop/ai assisted
3  Sneha   25   48000
4  Kiran   35   75000
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> &
python.exe "c:/Users/MYPC/OneDrive/Desktop/ai assisted
Element found at index: 2
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding>
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.